Definir a identidade do agente

Este módulo depende do seguinte:

- Ambiente do agente funcional do módulo 1
- Código do agente padrão criado por `adk create`
- Compreensão de que modelo + ferramentas + orquestração = agente

Como funciona seu primeiro agente

No módulo 1, você criou um agente funcional usando `adk create`. Quando você abriu o `agent.py`, este código apareceu:

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='Um agente assistente colaborativo.',
    instruction='Você é um assistente colaborativo.'
)
```

Você executou `adk web` para verificar que ele funciona. Ele é um sucesso. Mas o que os parâmetros de fato significam? Como você pode personalizar o agente para tarefas específicas? Pense neste módulo como uma forma de entender o componente "Modelo" do curso 1, que é o cérebro responsável pelo raciocínio do agente.

Quatro parâmetros principais

Referência: Documentação do ADK – Agente de LLM: Como definir a identidade e a finalidade do agente (em inglês)

Cada `LlmAgent` (também importado como `Agent`) é definido com quatro parâmetros principais. Vamos conhecer cada um deles.

---

1. `model` (obrigatório)

O que é: o modelo de linguagem grande (LLM) responsável pelo raciocínio e pela tomada de decisões do agente.

Exemplo:

```
model='gemini-2.5-flash'
```

O que ele faz:

- Determina a inteligência e os recursos do agente.
- Afeta o custo por solicitação e a velocidade de resposta.
- Modelos diferentes têm pontos fortes distintos (velocidade versus capacidade).

Modelos disponíveis:

- `gemini-2.5-flash`: rápido, eficiente e bom para a maioria das tarefas
- `gemini-2.5-pro`: mais eficiente e melhor para raciocínio complexo
- Consulte a documentação dos modelos para conferir a lista completa

Analogia: assim como acontece com o motor do seu carro, mecanismos mais potentes processam tarefas mais complexas, mas o custo pode ser mais alto.

Importante: ele é um **parâmetro obrigatório**. O agente não funciona sem especificar um modelo.

---

2. `name` (obrigatório)

O que é: um identificador de string exclusivo do agente.

Exemplo:

```
name='root_agent'
```

O que ele faz:

- Identifica o agente internamente no ADK.
- Essencial nos sistemas multiagente em que os agentes se referem uns aos outros.
- Usado para geração de registros, depuração e delegação de agentes.

Convenções de nomenclatura:

- ✅ Use letras minúsculas com sublinhados: `customer_support_agent`
- ✅ Seja descritivo: `data_analysis_agent`, `math_tutor_agent`
- ❌ Evite nomes reservados: `user`
- ❌ Não use camelCase: use `my_agent` e não `myAgent`

Trecho extraído da documentação do ADK (em tradução livre):

> "Todos os agentes precisam de um identificador de string exclusivo. Esse nome é essencial para operações internas, principalmente nos sistemas multiagente em que os agentes precisam se referir ou delegar tarefas uns aos outros."

Importante: ele é um **parâmetro obrigatório**. Cada agente precisa ter um nome.

---

3. `description` (opcional, recomendado para multiagente)

O que é: um resumo do que o agente faz.

Exemplo:

```
description='Responde a perguntas do usuário sobre a capital de um determinado país.'
```

O que ele faz:

- Usado por **outros agentes** para decidir se devem encaminhar tarefas para este agente.
- Auxilia nos sistemas multiagente em que os agentes delegam tarefas uns aos outros.
- **Não é usado pelo agente em si** para o próprio comportamento.

Trecho extraído da documentação do ADK (em tradução livre):

> "Esta descrição é usada principalmente por outros agentes de LLM para determinar se devem encaminhar uma tarefa para este agente. Ela deve ser específica o suficiente para diferenciá-lo dos pares."

Quando usar:

- ✅ Criação de sistemas multiagente com delegação
- ✅ Agentes que serão chamados por outros agentes
- ⚠️ Menos importante para aplicativos de agente único

Boas descrições:

- ✅ "Processa as consultas de faturamento dos clientes e as atualizações de pagamento"
- ✅ "Analisa dados de vendas e gera relatórios de performance semanais"
- ✅ "Ajuda estudantes a aprender álgebra com orientação pelas etapas de solução de problemas"
- ❌ "Agente de faturamento" (muito vago)
- ❌ "Auxiliar" (não é específico o suficiente)

---

4. `instruction` (crítico, mas opcional)

O que é: o plano comportamental que orienta como o agente age e responde.

Exemplo:

```
instruction="""Você é um assistente colaborativo que informa a hora atual nas cidades. Use a ferramenta 'get_current_time' tool para esta finalidade."""
```

O que ele faz:

- Define a personalidade e o estilo de comunicação do agente.

- Especifica a tarefa ou meta principal do agente.
- Estabelece limites e restrições em relação ao comportamento.
- Orienta sobre quando e como usar as ferramentas (abordado no módulo 3).
- Define o formato de saída.

Trecho extraído da documentação do ADK (em tradução livre):

> "Possivelmente, o parâmetro instruction é o mais importante para definir o comportamento de um LlmAgent. Ele informa ao agente qual é a tarefa ou meta principal, a personalidade ou persona, as restrições de comportamento, como e quando usar as ferramentas."

Dicas de instruções eficazes (extraídas da documentação do ADK):

- **Claras e específicas**: evite ambiguidades e indique claramente as ações e os resultados desejados
- **Usar markdown**: melhore a legibilidade das instruções complexas usando títulos, listas etc.
- **Incluir exemplos (few-shot)**: para tarefas complexas ou formatos de saída específicos, inclua exemplos
- **Orientar o uso de ferramentas**: não apenas liste as ferramentas, explique quando e por que o agente deve usá-las

---

`description` e `instruction`: a principal diferença

É fundamental entender o seguinte:

Parâmetro	Público-alvo	Finalidade	Exemplo
`description`	**Outros agentes**	"Devo encaminhar esta tarefa para cá?"	"Processa as consultas sobre faturamento"
`instruction`	**Este agente**	"Como devo me comportar?"	"Você é um especialista em faturamento que…"

Exemplo concreto:

```
# Sistema multiagente com 3 agentes
billing_agent = Agent(
    model='gemini-2.5-flash',
    name='billing_agent',
    # OUTROS agentes leem isto para decidir se devem delegar aqui
    description='Processa as consultas de faturamento dos clientes e os pagamentos',
    # ESTE agente lê isto para saber como se comportar
    instruction="""Você é especialista em faturamento.

    Quando ajudar os clientes:
    1. Tenha empatia e paciência
```

```
    2. Explique as cobranças com clareza
    3. Use a ferramenta billing_lookup para verificar os detalhes da conta
    4. Nunca prometa reembolsos sem aprovação do gerente

    Mantenha sempre um tom profissional e solícito."""
)

support_agent = Agent(
    model='gemini-2.5-flash',
    name='support_agent',
    description='Processa as perguntas gerais de suporte ao cliente',
    instruction="""Você é um agente de suporte ao cliente.

    Se a pergunta for sobre faturamento, transfira para o billing_agent.
    Caso contrário, ajude o cliente diretamente."""
)
```

Neste exemplo:

- O `support_agent` lê a **descrição** do `billing_agent` para decidir: "Esta é uma pergunta sobre faturamento?".
- O `billing_agent` lê a própria **instrução** para saber: "Como resolver as perguntas sobre faturamento?"

Convenção de nomenclatura de `root_agent`

Referência: Guia de início rápido do Python para ADK (em inglês)

Você deve ter notado que a variável do agente se chama `root_agent`:

```
root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',  # Nome interno (dentro do Agent)
    # ...
)
```

Por que `root_agent`?

As ferramentas de linha de comando do ADK procuram uma variável Python chamada `root_agent` como ponto de entrada para o sistema do agente. Trata-se de uma convenção que permite ao ADK descobrir e executar o agente.

Trecho extraído da documentação do ADK (em tradução livre):

> "O arquivo agent.py contém uma definição de root_agent, que é o único elemento obrigatório de um agente do ADK."

O nome interno pode ser diferente do nome da variável?

Sim. O parâmetro `name` em `Agent()` é separado do nome da variável:

```
# Nome interno do ADK (usado para geração de registros, delegação)
my_specialized_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',  # Usado internamente pelo ADK
    description='Ajuda estudantes com álgebra',
    instruction='Você é um orientador de matemática paciente...'
)

# Nome da variável que as ferramentas do ADK procuram (precisa ser root_agent)
root_agent = my_specialized_agent
```

Regra importante: sempre atribua o agente principal a uma variável chamada `root_agent` para que as ferramentas do ADK o encontrem.

No módulo 3, você vai aprender como as ferramentas do ADK usam o `root_agent` para executar o agente de várias maneiras.

Como escrever instruções eficazes

Referência: Documentação do ADK: Como orientar o agente: instruções (em inglês)

As instruções orientam o comportamento e a personalidade do agente. Neste curso, vamos usar exemplos simples para ajudar você a entender o conceito básico.

Abordagem simples para o curso 2:

```
instruction="Você é um orientador de matemática paciente. Ajude os estudantes a
resolver problemas de álgebra."
```

Essa instrução básica é suficiente para começar. Ela informa ao agente:

- O papel (orientador de matemática)
- O traço de personalidade (paciente)
- A tarefa (ajudar com álgebra)

Quer aprender padrões de instruções profissionais? No **curso 3: Como criar um comportamento melhor para o agente**, você vai aprender técnicas avançadas como:

- Estruturas de instrução com várias seções
- Definições de persona e limites
- Exemplos few-shot
- Padrões de instrução prontos para produção

Por enquanto, vamos nos concentrar no funcionamento das instruções básicas.